

```
import numpy as np

def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))
```

```
class KMeans:
    def __init__(self, k=2, max_iters=100):
        self.k = k
        self.max_iters = max_iters

    def fit(self, X):
        # Pick random initial centroids from the dataset
        idx = np.random.choice(X.shape[0], self.k, replace=False)
        self.centroids = X[idx]

        for _ in range(self.max_iters):
            # Assign clusters
            self.labels = np.array([np.argmin([euclidean_distance(x, c) for c in
self.centroids]) for x in X])

            # Recompute Centroids
            new_centroids = np.array([X[self.labels == i].mean(axis=0) if
len(X[self.labels == i]) > 0 else self.centroids[i] for i in range(self.k)])
            if np.all(self.centroids == new_centroids):
                break
            self.centroids = new_centroids
        return self.labels

    def silhouette_score_simple(X, labels):
        # Simple evaluation: Mean within-cluster vs closest-cluster distance
        scores = []
        for i, x in enumerate(X):
            c_idx = labels[i]
            same_cluster = X[labels == c_idx]
            other_cluster = X[labels != c_idx]

            if len(same_cluster) > 1:
                a = np.mean([euclidean_distance(x, original) for original in same_cluster
if not np.array_equal(x, original)])
            else:
                a = 0

            if len(other_cluster) > 0:
                b = np.min([np.mean([euclidean_distance(x, o) for o in X[labels == c]])
for c in set(labels) if c != c_idx])
            else:
                b = 0

            if max(a, b) == 0:
                scores.append(0)
            else:
                scores.append((b - a) / max(a, b))
        return np.mean(scores)
```

```
# Example Usage
if __name__ == "__main__":
    X = np.array([[1, 2], [1, 4], [1, 0], [10, 2], [10, 4], [10, 0]])

    km = KMeans(k=2)
    labels = km.fit(X)

    score = silhouette_score_simple(X, labels)
    print("Assigned Labels:", labels)
    print("Calculated Silhouette Validation Score:", score)
```

Program 10: K-Means Clustering and Cluster Validation (Second Run)

Assigned Labels: [0 0 0 1 1 1]

Calculated Silhouette Validation Score: 0.7133477791749615

Shivani kumari 24becs105